CSE 755 – Practice Problems with Solutions

Problem 1: Mixture responsibility

This is a two-component 1D Gaussian mixture. Both components share the variance $\sigma^2 = 1$:

$$p(z = 1) = \pi = 0.6, \qquad p(z = 2) = 0.4, \qquad p(x \mid z = k) = \mathcal{N}(x; \mu_k, 1),$$

with $\mu_1 = 0$ and $\mu_2 = 3$.

(A) Write $p(x)$ and the responsibility $r_1(x) = p(z = 1 \mid x)$.

(B) Evaluate $r_1(1)$ numerically. Leave the two Gaussian densities visible before dividing.

$$\mathcal{N}(x; \mu, 1) = \frac{1}{\sqrt{2\pi}} e^{-(x-\mu)^2/2}.$$

Solution

(A)

$$p(x) = 0.6\,\mathcal{N}(x; 0, 1) + 0.4\,\mathcal{N}(x; 3, 1)$$

$$r_1(x) = \frac{0.6\,\mathcal{N}(x; 0, 1)}{0.6\,\mathcal{N}(x; 0, 1) + 0.4\,\mathcal{N}(x; 3, 1)} = \frac{1}{1 + \frac{2}{3} e^{3x - 4.5}}$$

(B)

$$\mathcal{N}(1; 0, 1) = \frac{e^{-0.5}}{\sqrt{2\pi}} \approx 0.24197, \qquad \mathcal{N}(1; 3, 1) = \frac{e^{-2}}{\sqrt{2\pi}} \approx 0.05399$$

$$r_1(1) = \frac{0.6(0.24197)}{0.6(0.24197) + 0.4(0.05399)} = \frac{0.14518}{0.16678} \approx \boxed{0.8705}$$

Explanation

The setup. Think of the mixture as a generator that makes a number x in two steps. First it picks a component z: component 1 with probability 0.6, component 2 with probability 0.4. Then it draws x from that component's bell curve. Component 1's bell curve is centred at 0, and component 2's is centred at 3. When we see a generated x, we never see which component made it. That hidden choice z is called the *latent variable*.

Why $p(x)$ looks like that. There are two ways x could have been produced: through component 1 or through component 2. The probability of each way is "probability of picking that component" times "probability that the component produces x". Adding the two ways gives the total probability

of x. This is the *law of total probability*, applied by summing out z. As a picture, $p(x)$ is a curve with two bumps: a taller one at 0, because it has weight 0.6, and a shorter one at 3.

Why $r_1(x)$ looks like that. The responsibility asks a reverse question: *given that we saw x, how likely is it that component 1 made it?* Bayes' rule answers it. Take the "component 1 way" of producing x (the numerator) and divide by the total over all ways (the denominator, which is $p(x)$). The result is component 1's share of the credit for x. It is always between 0 and 1, and the two responsibilities add up to 1: $r_1(x) + r_2(x) = 1$.

The simplified form. Divide the top and bottom of the fraction by $0.6\,\mathcal{N}(x; 0, 1)$. Both Gaussians have the same variance, so their $\frac{1}{\sqrt{2\pi}}$ factors cancel. What remains is a ratio of exponentials:

$$\frac{\mathcal{N}(x; 3, 1)}{\mathcal{N}(x; 0, 1)} = \exp\Big(\tfrac{x^2-(x-3)^2}{2}\Big) = e^{3x-4.5}.$$

This gives $r_1(x) = 1/(1 + \frac{2}{3}e^{3x-4.5})$, which is a sigmoid (S-shaped) curve. For very negative x it is close to 1: those points almost surely came from component 1. For large x it is close to 0: those points almost surely came from component 2.

Part (B), step by step. First, plug $x = 1$ into each bell curve. The point is 1 unit from the centre of component 1, so its density is fairly high (0.242). It is 2 units from the centre of component 2, so its density is much lower (0.054). Next, multiply each density by its component's prior weight: 0.145 and 0.022. Finally, divide component 1's part by the total.

What the answer means. $r_1(1) \approx 0.87$ means there is about an 87% chance that $x = 1$ came from component 1, and about a 13% chance that it came from component 2. Two things push the answer toward component 1: the point is closer to its centre, and that component has the larger prior weight. As a check, the simplified formula gives $1/(1 + \frac{2}{3}e^{-1.5}) = 0.8705$, the same answer.

Why this matters. In the EM algorithm, this calculation is the *E-step*. Each data point is not forced into one cluster. Instead, it gets a "soft" assignment, such as 87% to cluster 1 and 13% to cluster 2.

Problem 2: Exact likelihood and bits per dimension

An autoregressive model assigns the following probabilities to the four true bits of a binary image:

$$0.5, \quad 0.25, \quad 0.8, \quad 0.4.$$

(A) Compute $p(\mathbf{x}) = \prod_{i=1}^{4} p(x_i \mid x_{<i})$.

(B) Compute $-\log_2 p(\mathbf{x})$ and $\text{bpd}(\mathbf{x}) = -\frac{1}{4}\log_2 p(\mathbf{x})$.

(C) A uniform binary model uses 1 bit per coordinate. Did this model beat that baseline?

Solution

(A)

$$p(\mathbf{x}) = 0.5 \times 0.25 \times 0.8 \times 0.4 = \boxed{0.04}$$

(B)

$$-\log_2 0.04 = \log_2 25 \approx \boxed{4.644 \text{ bits}}, \qquad \text{bpd} = \frac{4.644}{4} \approx \boxed{1.161 \text{ bits/dim}}$$

(C) Uniform baseline: $0.5^4 = 0.0625$, which is 1 bit/dim. Since $1.161 > 1$, **the model did not beat the baseline.**

Explanation

How an autoregressive model generates an image. The model generates the image one pixel at a time, in a fixed order. For each pixel, it looks at all the pixels already generated and outputs a probability for the next one. The numbers in the problem are the probabilities the model gave to the *true* value of each pixel: 0.5 for pixel 1, then 0.25 for pixel 2 given pixel 1, and so on.

Part (A): why we multiply. The probability of the whole image is the probability of getting pixel 1 right, *and then* pixel 2 right given pixel 1, *and then* pixel 3, and so on. "And then" means multiply. This is the *chain rule of probability*, and it is exact. This is a key strength of autoregressive models: they give the true likelihood $p(\mathbf{x})$, not an approximation or a bound as a VAE does. The result, 0.04, means the model gives this exact image a 4% chance.

Part (B): why we take $-\log_2$. Probabilities of whole images get tiny very fast, so we work with logs instead. The quantity $-\log_2 p$ has a useful meaning: it is the number of *bits* needed to store or send that outcome with an ideal compression code. The log turns the product into a sum, so the total cost is just each pixel's cost added together:

Bit i	$p(x_i \mid x_{<i})$	$-\log_2 p$ (bits)
1	0.5	1.000
2	0.25	2.000
3	0.8	0.322
4	0.4	1.322
Total		4.644

Notice the pattern. When the model is confident and right (0.8), the pixel is cheap: only 0.32 bits. When the model is surprised (0.25), the pixel is expensive: 2 bits. In general, a lower probability means more bits.

Part (B): why we divide by 4. Bigger images need more bits in total, so total bits cannot fairly compare models on images of different sizes. Dividing by the number of pixels gives *bits per dimension* (bpd), the average cost of one pixel. Here it is 1.161. Lower bpd means a better model.

Part (C): the baseline. The simplest possible model knows nothing and says 50/50 for every pixel. Each pixel then costs exactly $-\log_2 0.5 = 1$ bit, so its bpd is 1. A useful model should do better than this, which means a bpd below 1.

Why this model lost. Our model scores 1.161, which is *worse* than random guessing. Look at the table. On pixel 3 it saved bits (0.32 instead of 1). But on pixel 2 (0.25) and pixel 4 (0.4) it gave the true value less than 50%, so it was confidently wrong. Those pixels cost 2 and 1.32 bits, more than a coin flip would. The extra cost from those two mistakes is larger than the saving on pixel 3. The likelihoods tell the same story: $0.04 < 0.0625$.